

* Binary is enough to represent each and every possible thing be it audio, visuals etc. Image, sound, video, text, computer program, state of machine etc. so ~~at~~ most times our alphabet will have 2 characters (sometimes 3, but even 2 are enough).

All 5 things are needed to define a finite automata. Only 1 restriction on & that it has to be finite and not be null.

Transition Function:

when the machine is in a particular state and it reads a character and decides what to do. we can have a function, or a table to define this transition. mathematical

There will always be a starting state.

possible for none of the states to be final states, all of them are final, or final, more than one final $\rightarrow$ everything is possible.

definition to Machine } both are
or } possible
Machine to definition

once you draw/define the machine once, but after that, during its execution you can't change it.

Having no final state is a matter of mathematical completeness.

—X —X —X —X —X —
if M accepts A, then A is lang of M.

A is a lang
lang is a set of strings. ^{be2} it contains different strings but A isn't any arbitrary string. It is a string that is accepted by some machine M.
so A is the lang of machine M.

Some machine could accept infinitely many languages.

—X —X —X —X —X —

A machine can accept many strings but can have only 1 lang.

A language is a set of acceptable ~~states~~ strings.

Finite Automata with 2 states.

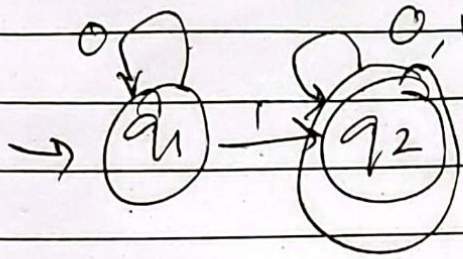
Date: _____

Eg 1

$A = \{w : w \text{ contains at least one } 1\}$

$A = \{0, 1\}$

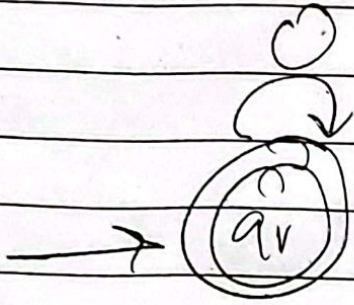
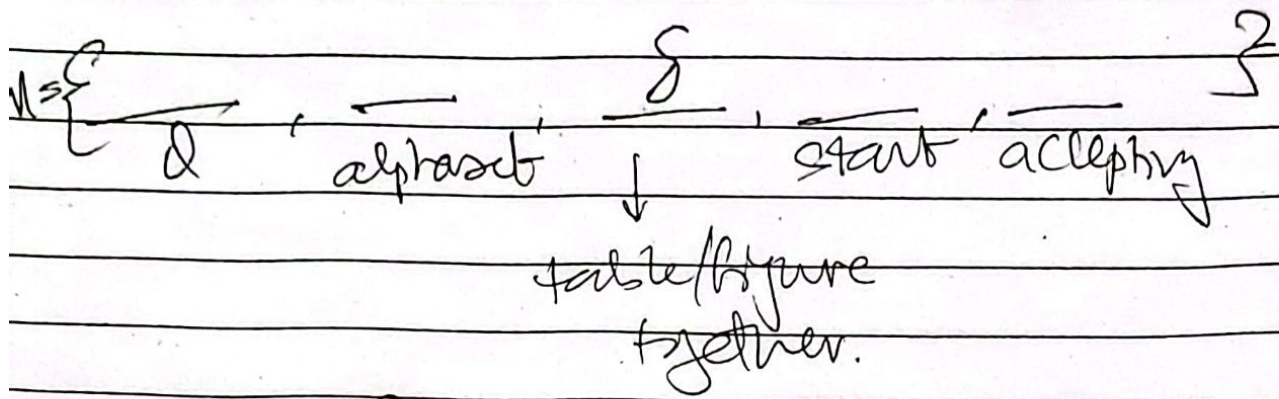
$Q = q_1, q_2$



2 states. An accepting and rejecting state. As long as input is initially 0, I don't accept it. As soon as I go to accepting, once it's there I keep there regardless of 0 & 1.

Finite Automaton does not remember what happened in the past. It can only remember the present and decide the next state.

You cannot look at that Mall and see how you got there. There is no past history. You cannot go to a particular state in any possible way.



(→ it also accepts an empty string.
 as soon as you start it, it is
 accepting and stays there for
 an empty string. also accepts
 anything that ends on 0.

Regular Language

Any lang accepted by finite automaton is called regular lang.

A regular lang will have a finite automaton that will accept it.

Both def need to be fulfilled.

UNION:

whatever is in both will appear once, whatever is in either one appears once.

can find unions of set and lang is a set so can have union of lang.

Concatenation:

String 1 = a ; String 2 = ab

String 3 = aab } concatenation of String 1 . String 2

String 4 = aba } concatenation of String 2 . String 1

concatenation of strings matters.

in Java notation, concatenation
shown using +

we have a language A such

$$A = \{a, ab, aab\}$$

and lang B such as

$$B = \{b, ba, bba\}$$

$AB =$ concatenation of languages
A and B

into
concatenations
either A.B and
usually simply AB

$$C = AB = \{ \text{~~a~~, ~~ab~~, ~~ba~~},$$

$ab, aba, abba, \dots$ // coming from

a of A

$ab, abba, abbaa, \dots$ // coming from
 ab of A

$aab, aabba, aabbaa, \dots$ // coming from
 aab of A

if there is a repetitive element then remove it. (removing one abba as repeating).

so final version of C is

$C = \{ ab, aba, abba, abb, abbaa, aabb, aabba, aabbaa \}$

Star or Kleen-Star Operation:

Kleen of a string:

a is a string.

then

kleen of $a = a^* = \{ \epsilon, a, aa, aaa, \dots \}$

kleen of a string creates infinitely many strings including length 0.

Star guarantees a string of length 0 (That is empty string epsilon will 100% be there).

eg: $str = ab$

$str^* = \{ \epsilon, ab, abab, ababab, \dots \}$

0 copies of ab , 1 copies of ab , 2 copies of ab and so on...

whenever you put a star to anything.
it becomes a language even if it is a
string.

Not every set has an epsilon but
every ~~set~~ Kleene star will have an
epsilon.

languages:

$$A = \{a, b\}$$

~~$$A^* = \{\epsilon, a, b, aa, bb, \dots\}$$~~

$$A^* = \{\epsilon, a, aa, aaa, \dots, b, bb, bbb, \dots, ab, aab, aabaa\}$$

any no. of a and b and possibilities
including epsilon and copies of itself
a, b

Theorem 1:

class of regular lang is closed under

- union operation.
- concatenation operation.

"closed under" means.

eg ~~set of~~ natural numbers is closed under addition but not subtraction.

integers.
set of ~~real~~ ~~integers~~ are closed under addition, subtraction and multiplication but not division.

if you take 2 regular lang (small or big doesn't matter) then:

you can still construct a finite automata for both the union operation and concatenation operation of the two languages.

examples cannot replace
a proof

conjecture is a statement that some
people think is true but still can't
prove it.

Now do we prove a lang is regular?
→ you can make a DFA or automaton
for it

proving union of 2 regular ~~lang~~ langs
is again a regular lang.

Sketch of a proof:

① Having an arbitrary regular lang A
then there must be some machine
 M_A that recognizes A .

then M_A will have some states

② for sake of simplicity maybe there
the two langs have same alphabet
without loss of generality.

Both these langs will have finite
cutoff at q .

Tally this page

Date: _____

A new machine C w³

State and transition function are most important.

no. of states of A union B is equal to Cartesian product of this new machine

Transition functions.

All the states in my new machine C has ~~2~~ 2 internal ~~states~~ sub states. so every state h.